

Python Basics — Quick Revision Notes

A concise reference for recalling Python fundamentals before moving into Generative AI development.

1. Variables

Concept

A variable is a name that refers to a value. Python determines the type automatically.

Example

```
name = "Gowtham"  
age = 22  
salary = 45000.50  
is_learning = True
```

Useful operations

```
print(name)  
print(type(age))  
x, y = 10, 20  
x, y = y, x
```

Remember

Python is dynamically typed: the same variable can later refer to a value of another type.

2. Conditions

Concept

Conditions let a program make decisions using if, elif, and else.

Example

```
age = 22  
if age >= 18:  
    print("Adult")  
else:  
    print("Minor")
```

Multiple conditions

```
marks = 82  
if marks >= 90:  
    grade = "A+"  
elif marks >= 75:  
    grade = "A"  
else:  
    grade = "B"
```

Operators

Comparison: ==, !=, >, <, >=, <=
Logical: and, or, not
Membership: in, not in

3. Loops

Concept

Loops repeat a block of code.

for loop

```
for i in range(5):  
    print(i)
```

Iterating a list

```
skills = ["Java", "Python", "SQL"]  
for skill in skills:  
    print(skill)
```

while loop

```
count = 1  
while count <= 3:  
    print(count)  
    count += 1
```

Control

break stops the loop. continue skips the current iteration. pass is a placeholder.

4. Basic Functions

Concept

A function is a reusable block of code defined with def.

Example

```
def greet(name):  
    return f"Hello, {name}!"  
  
message = greet("Gowtham")  
print(message)
```

Default argument

```
def power(number, exponent=2):  
    return number ** exponent  
  
print(power(5))  
print(power(5, 3))
```

Key points

Parameters receive input. return sends a value back. Arguments are values passed to a function.

5. Basic Lists and Dictionaries

Lists

```
numbers = [10, 20, 30]  
numbers.append(40)  
numbers[0] = 15  
print(numbers[1:3])
```

Common list operations

append(), extend(), insert(), remove(), pop(), sort(), reverse(), len()

Dictionaries

```
student = {"name": "Gowtham", "age": 22, "branch": "IT"}
print(student["name"])
student["age"] = 23
student["city"] = "Vijayawada"
```

Useful dictionary operations

keys(), values(), items(), get(), update(), pop()

Iteration

```
for key, value in student.items():
    print(key, value)
```

6. Basic OOP

Concept

A class is a blueprint; an object is an instance of a class.

Example

```
class Student:
    def __init__(self, name, branch):
        self.name = name
        self.branch = branch

    def introduce(self):
        return f"{self.name} - {self.branch}"

s = Student("Gowtham", "IT")
print(s.introduce())
```

Remember

self refers to the current object. __init__ initializes object attributes. Methods are functions inside a class.

Core OOP ideas

Encapsulation, inheritance, polymorphism, and abstraction.

7. Basic String Operations

Concept

Strings are sequences of characters and are immutable.

Example

```
text = "Generative AI"
print(text.lower())
print(text.upper())
print(text.strip())
print(text.replace("AI", "Intelligence"))
```

Indexing/slicing

```
text = "Python"
print(text[0])
print(text[-1])
print(text[0:3])
```

```
print(text[::-1])
```

Useful methods

split(), join(), find(), count(), startswith(), endswith(), replace(), strip()

f-strings

```
name = "Gowtham"  
topic = "Generative AI"  
print(f"{name} is learning {topic}.")
```

Quick Recall Cheat Sheet

Topic	Remember these
Variables	assignment, dynamic typing, type()
Conditions	if / elif / else, comparisons, and/or/not
Loops	for, while, range(), break, continue
Functions	def, parameters, arguments, return
Lists	ordered, mutable, indexing, slicing, append/pop
Dictionaries	key-value pairs, get(), items(), keys(), values()
OOP	class, object, __init__, self, methods
Strings	immutable, indexing, slicing, split/join, f-strings

GenAI connection: These basics will later support data processing, API calls, document processing, RAG pipelines, model interaction, and backend development.